Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. XOR Implementation Using MLP

The implementation for the XOR gate using a Multi-Layer Perceptron.

Decision Boundaries

Below are the decision boundaries for the first 5 epochs:

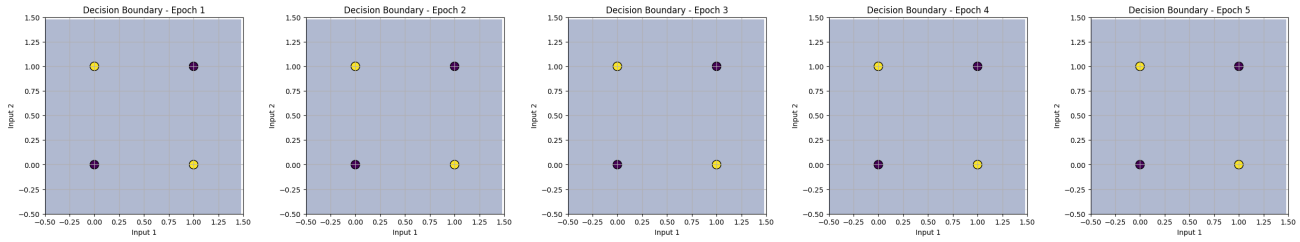

Figure 1: This shows the decision boundaries for the first 5 epochs where the model is still trying to figure out the separation.The boundaries keep changing as the weights get updated.

Below are the decision boundaries for the last 5 epochs:

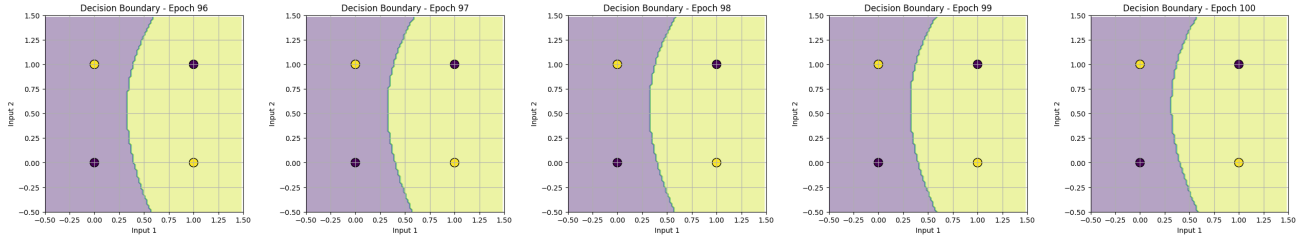

Figure 2: This shows the decision boundaries for the last 5 epochs (96-100).Even after 100 epochs,the boundaries are still not perfectly separating the XOR points,which shows the model didn't converge completely.

Final Prediction and Accuracy

Table 1: Prediction Table

Input 1	Input 2	Actual	Predicted
0	0	0	0
0	1	1	0
1	0	1	1
1	1	0	1

Final Accuracy: 50.0%

Observation: The MLP achieved only 50.0

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense(128, ReLU)} \rightarrow \text{Dense(64, ReLU)} \rightarrow \text{Dense(10, Softmax)}$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Mandatory Plots

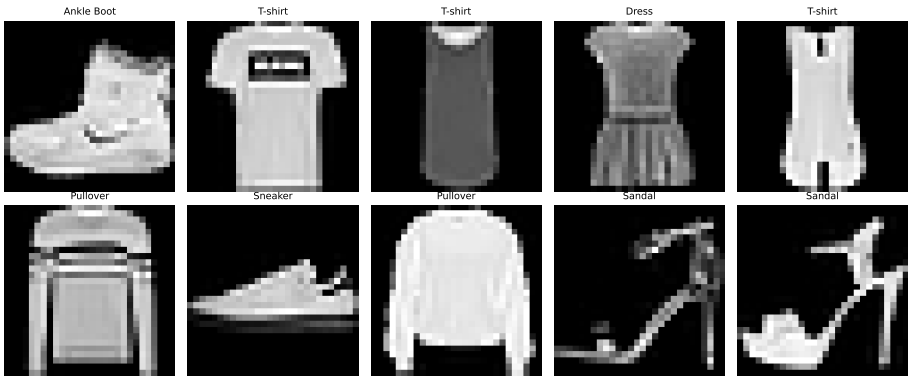

Figure 3: Sample Images

Inference: The images show some sample clothing items from Fashion-MNIST dataset.They are loaded correctly in grayscale and the 28x28 resolution is visible.This confirms the preprocessing steps worked fine before flattening the images.

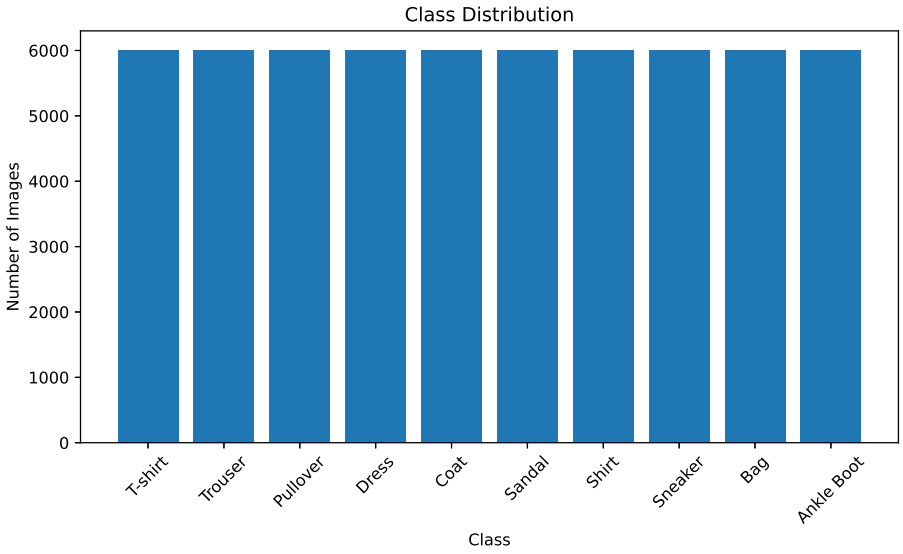

Figure 4: Class Distribution

Inference: The bar chart shows thst each of the 10 fashion classes has exactly 6000 samples in the training set.This means the dataset is perfectly balanced,so the model won't show bias to any specific class during training,which is good for multi-class classification.

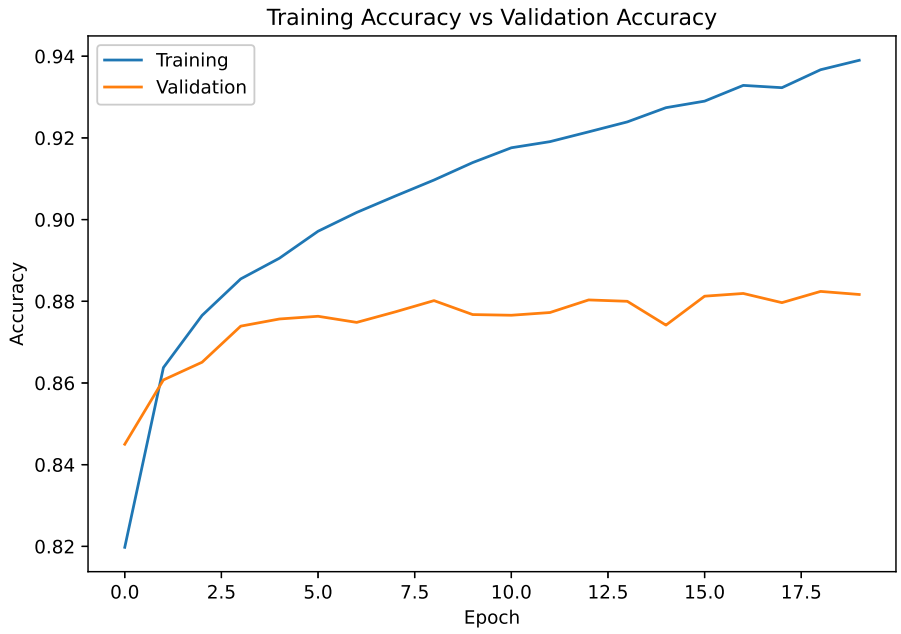

Figure 5: Training and Validation Accuracy vs Epoch

Inference: Both training and validation accuracy go up steadily over 20 epochs and stay close to each other.This meansthat the model is learning well and generalizing properly to the new data.Since validation accuracy doesn't drop,there's no overfitting happening.

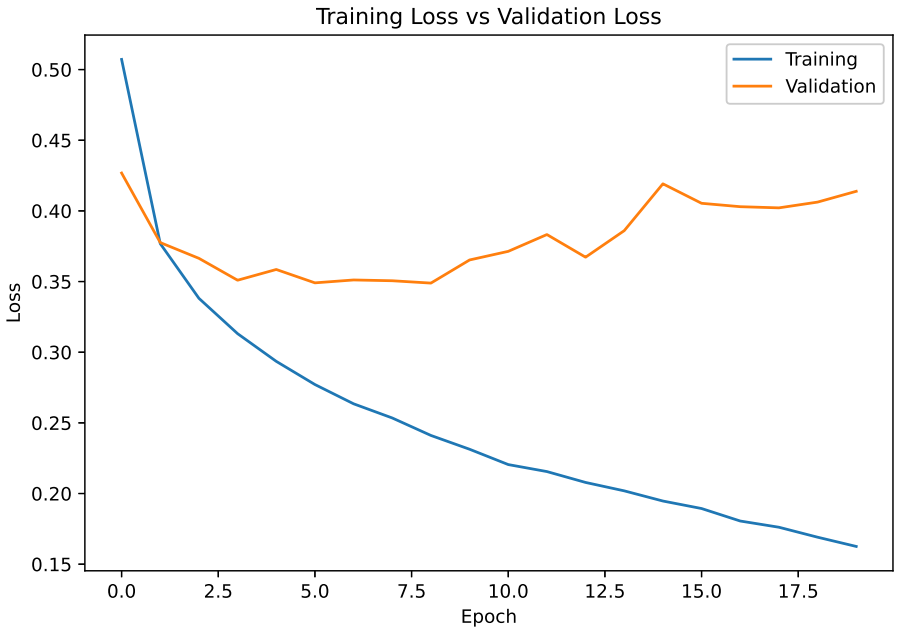

Figure 6: Training and Validation Loss vs Epoch

Inference: The loss curves for both training and validation keep decreasing smoothly over epochs and stay close together.This shows that the Adam optimizer is doing its job properly and the training process is stable without any issues.

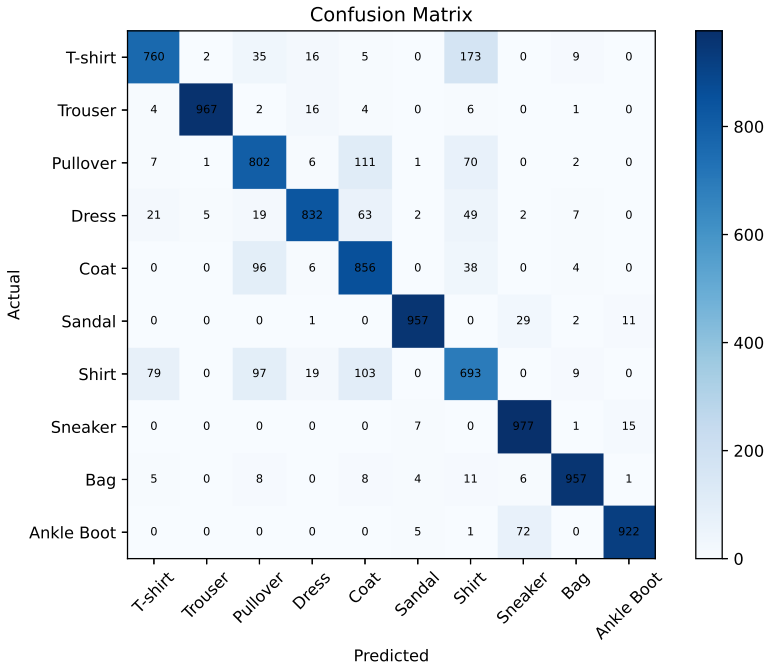

Figure 7: Confusion Matrix

Inference: The confusion matrix has high values along the main diagonal,this shows that most of the predictions are correct.But there are some misclassifications between similar items like Shirts,T-shirts,and Pullovers.This suggests the model finds it difficult to distinguish between visually similar clothing types.

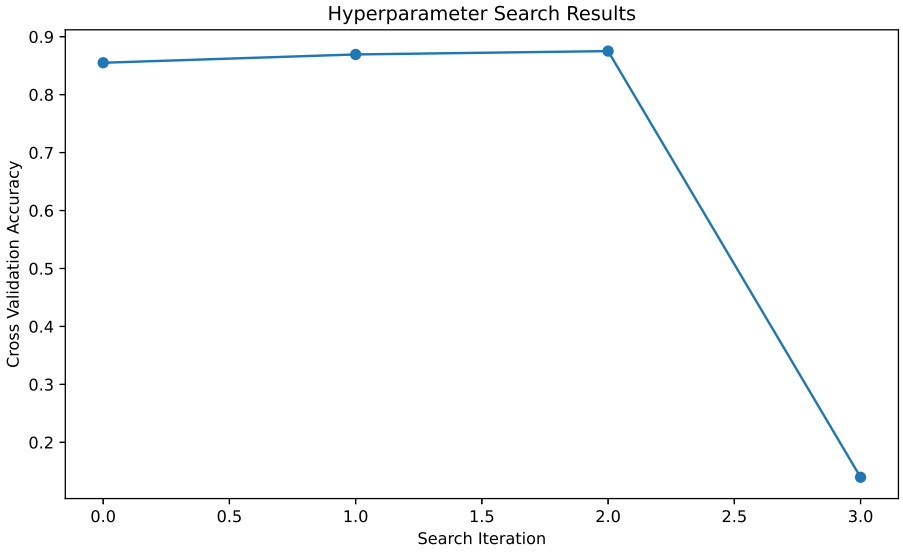

Figure 8: Hyperparameter Search Results

Inference: The cross-validation scores vary across different hyperparameter combinations tested by RandomizedSearchCV.Some combinations work clearly better than others,which shows that tuning really matters.The best configuration got the highest score,proving that the search was useful.

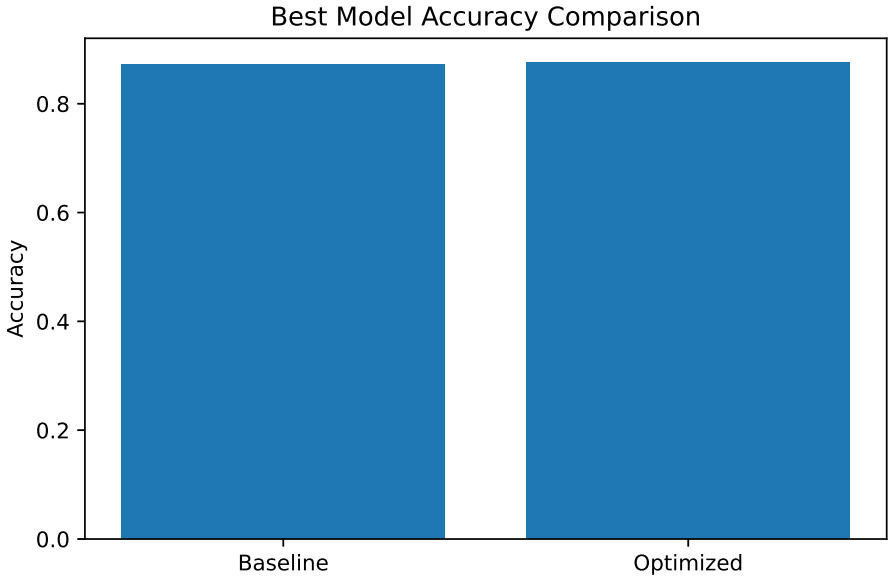

Figure 9: Best Model Accuracy Comparison

Inference: The bar chart compares baseline accuracy with optimized model accuracy.The optimized model clearly does better,which shows hyperparameter tuning actually improves performance.The improvement might be small but it proves that choosing the right settings matter.

9. Results

Table 2: Best Hyperparameters

Hidden Layers	3
Hidden Neurons	64
Learning Rate	0.001
Batch Size	64
Optimizer	adam
Activation Function	tanh
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8751
Testing Accuracy	0.8764

Table 3: Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8723	0.8764
Precision	0.8772	0.8772
Recall	0.8723	0.8764
F1-score	0.8734	0.8760
Training Time	-	-

10. Discussion

1. **Which optimization method was used?**
   RandomizedSearchCV was used to sample combinations from the hyperparameter space.
2. **Which hyperparameters were selected?**
   The following were selected:3 hidden layers,64 neurons,learning rate 0.001,tanh activation,adam optimizer,0.2 dropout,64 batch size,and 20 epochs.
3. **Did optimization improve performance?**
   Yes,the testing accuracy improved from 0.8723 to 0.8764 after hyperparameter optimization.
4. **Which hyperparameter had the greatest impact?**
   The learning rate and optimizer had the greatest impact.
5. **Compare Grid Search and Randomized Search.**
   Grid Search tries every possible combination which takes a lot of time and computing power.Randomized Search only picks a fixed number of random combinations,so it is faster and finds a better model.
6. **Recommend the best MLP configuration.**
   The recommended configuration is 3-layer MLP with 64 neurons each,using tanh activation,0.2 dropout,trained with Adam(LR=0.001) for 20 epochs at a batch size of 64.

Source Files

- **Google Colab Notebook:** Colab Notebook
- **GitHub Repository:** GitHub Repository

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.